

Project 1:

Iris Flower Species Prediction System using Machine Learning

Problem Definition

Identifying the species of an iris flower manually based on its physical measurements (sepal length, sepal width, petal length, petal width) can be time-consuming and prone to human error.

The task is to accurately classify iris flowers into their respective species based on physical measurements. Since multiple machine learning algorithms can solve this classification problem, selecting the most suitable model with the highest accuracy is a key challenge

Project Description

This project follows a comparative machine learning approach where multiple supervised classification models are trained and evaluated on the Iris dataset.

The workflow includes:

1. Training different supervised learning models
2. Evaluating their performance using accuracy
3. Selecting the best-performing model
4. Deploying the final model using a Streamlit web application

After comparison, Logistic Regression achieved the highest accuracy, and hence it was selected as the final model for deployment.

Tools & Libraries Used:

Programming Language:

- Python

Libraries:

- **Streamlit** → Web app development
- **Pandas** → Data preprocessing
- **NumPy** → Numerical computations
- **Scikit-learn** → Model training & evaluation
- **Joblib** → Model persistence

Technologies & Concepts Used

- Supervised Learning
- Model Comparison & Selection
- Classification Algorithms

- Logistic Regression
- Model Deployment (Streamlit)

Code Implementation:

```
import streamlit as st

import pandas as pd

import numpy as np

import joblib

from sklearn.datasets import load_iris

# Load the trained model

model = joblib.load('iris_model.pkl')

# Title

st.title("🌸 Iris Flower Species Prediction")

st.write("Enter flower measurements to predict the species.")

# User inputs

sepal_length = st.slider('Sepal length (cm)', 4.0, 8.0, 5.1)

sepal_width = st.slider('Sepal width (cm)', 2.0, 4.5, 3.5)

petal_length = st.slider('Petal length (cm)', 1.0, 7.0, 1.4)

petal_width = st.slider('Petal width (cm)', 0.1, 2.5, 0.2)

# Prepare input data

input_data = np.array([[sepal_length, sepal_width, petal_length, petal_width]])

# Prediction

if st.button('Predict'):

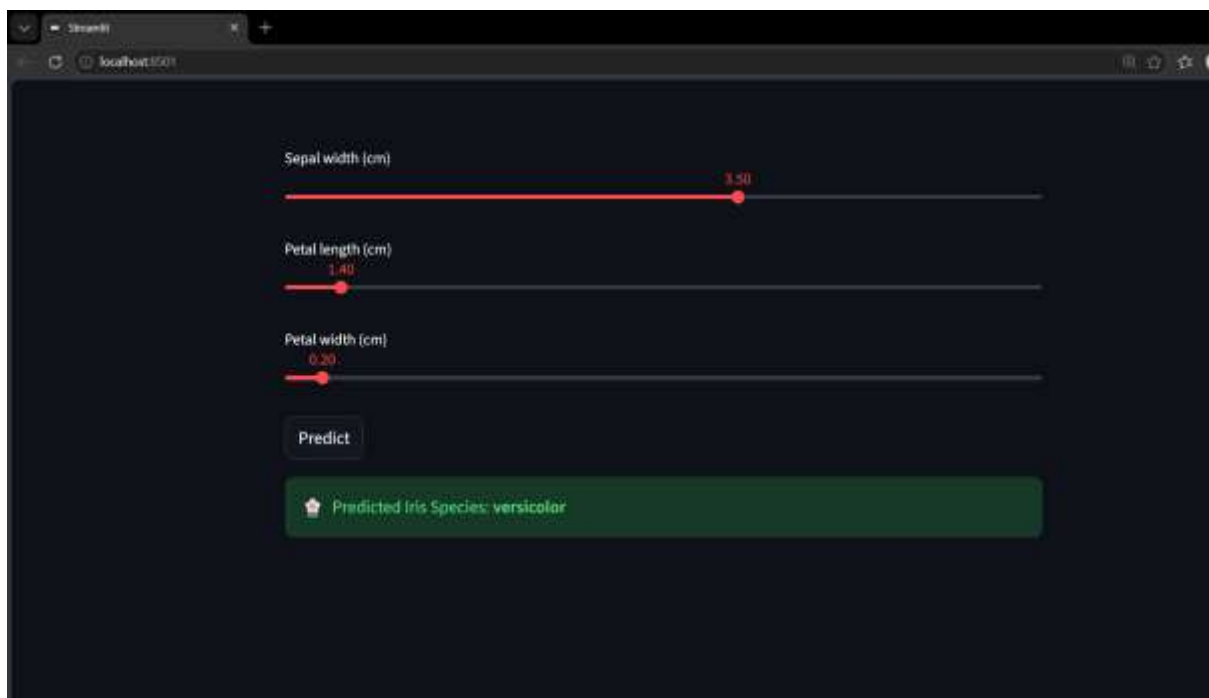
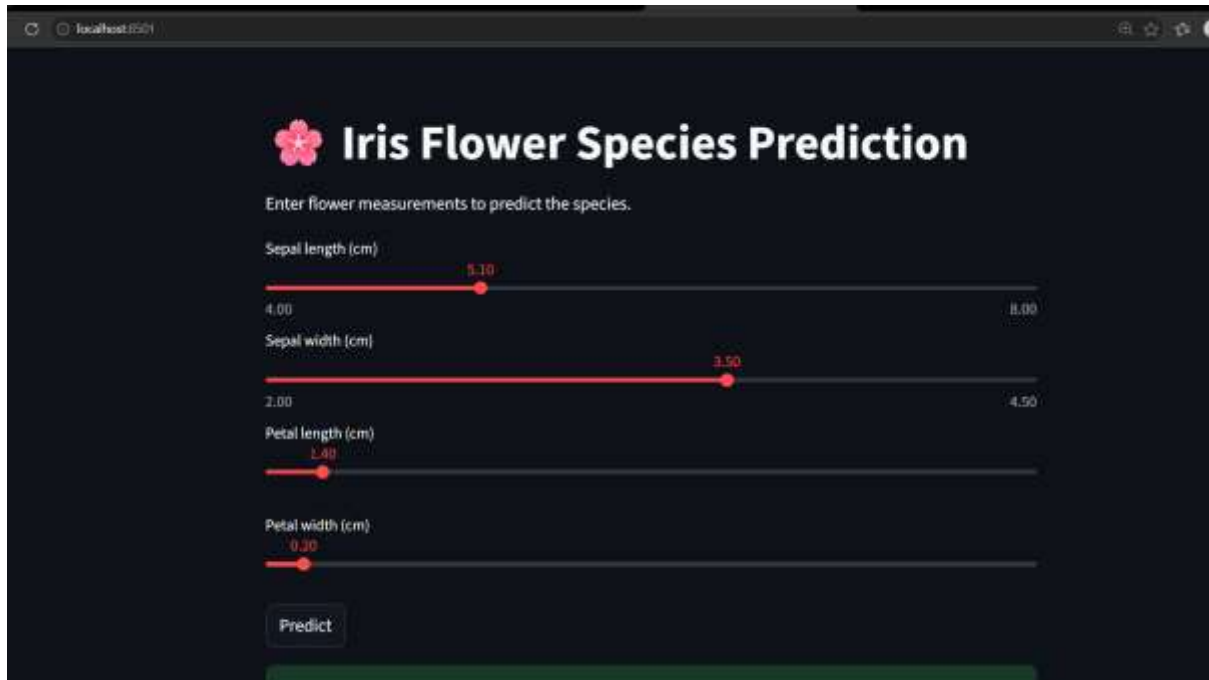
    prediction = model.predict(input_data)

    iris = load_iris()
```

```
predicted_class = iris.target_names[prediction[0]]
```

```
st.success(f"🌸 Predicted Iris Species: **{predicted_class}**")
```

Output:



Project 2:

Breast Cancer Prediction

Problem Definition

Early detection of breast cancer is crucial for effective treatment and improved survival rates. However, manually analyzing medical data to determine whether a tumor is malignant (cancerous) or benign (non-cancerous) can be complex and error-prone.

The problem is to develop a machine learning-based system that can accurately classify tumors using patient medical features.

Project Description

This project implements a machine learning classification system that predicts whether a tumor is malignant or benign based on medical input features.

The approach includes:

1. Training multiple supervised learning models
2. Comparing their performance using accuracy
3. Selecting the best-performing model
4. Deploying the model using a Streamlit web application

After evaluation, Logistic Regression achieved the highest accuracy and was selected as the final model.

The system also applies feature scaling (standardization) to improve model performance and ensure consistent predictions.

Tools & Libraries Used

Programming Language:

- Python

Libraries:

- **Streamlit** → Web application interface
- **NumPy** → Numerical operations
- **Scikit-learn** → Model training, scaling, evaluation
- **Pickle** → Saving and loading model, scaler, and features

Code Implementation:

```
import streamlit as st
```

```
import numpy as np
```

```
import pickle
```

```
# Load saved files

model = pickle.load(open("cancer_model.pkl", "rb"))
scaler = pickle.load(open("scaler.pkl", "rb"))
features = pickle.load(open("features.pkl", "rb"))

# Title

st.title("🧬 Breast Cancer Prediction App")
st.write("Enter the key medical values to predict tumor type.")

# Input fields

input_data = []

for feature in features:
    value = st.number_input(f"{feature}", value=0.0)
    input_data.append(value)

# Convert to array

input_data = np.array(input_data).reshape(1, -1)

# Apply scaling

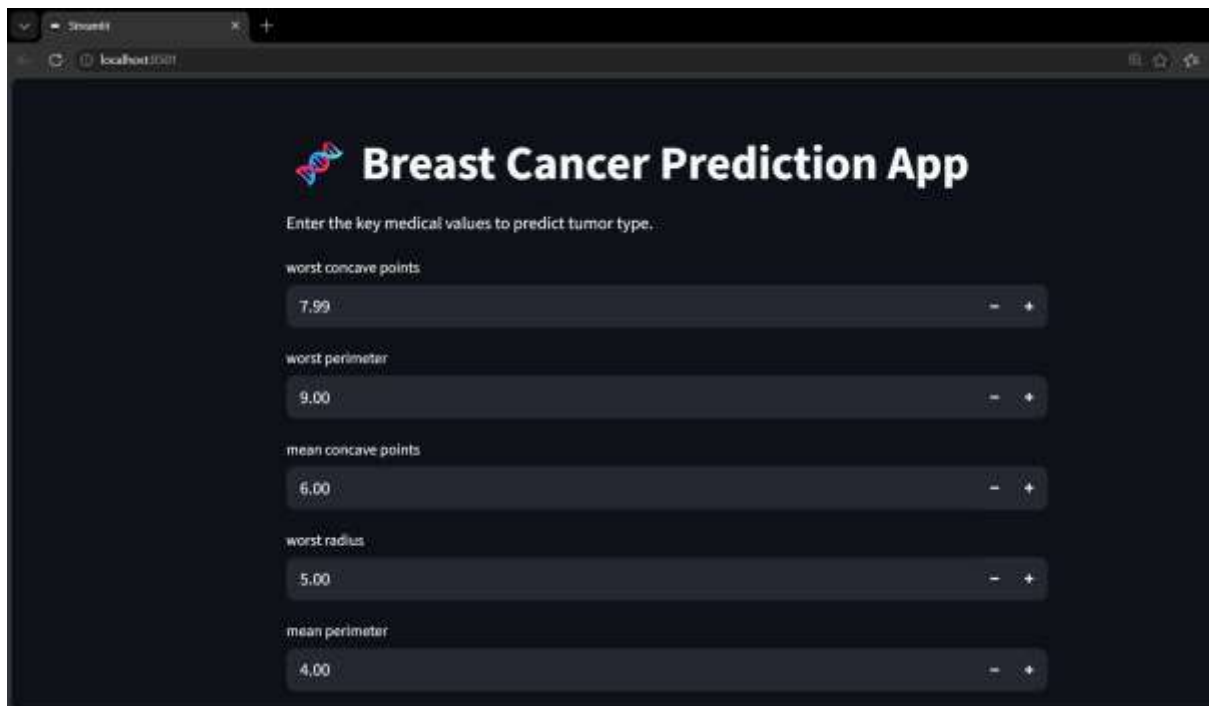
input_data = scaler.transform(input_data)

# Prediction

if st.button("Predict"):
    prediction = model.predict(input_data)

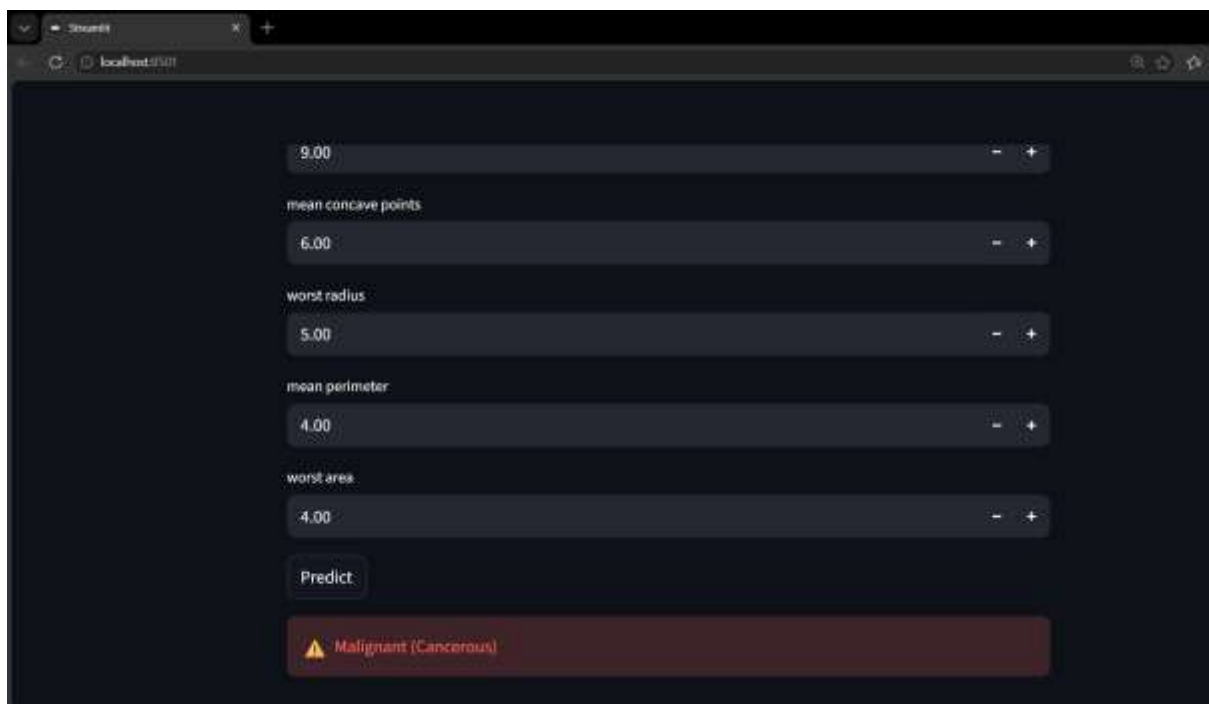
    if prediction[0] == 0:
        st.error("⚠️ Malignant (Cancerous)")
    else:
        st.success("✅ Benign (Non-Cancerous)")
```

Output:



A screenshot of a web browser displaying the "Breast Cancer Prediction App". The browser's address bar shows "localhost:5001". The app has a dark theme. At the top, there is a logo of a DNA double helix and the title "Breast Cancer Prediction App". Below the title, a subtitle reads "Enter the key medical values to predict tumor type." There are five input fields, each with a label and a value, and a minus/plus control on the right:

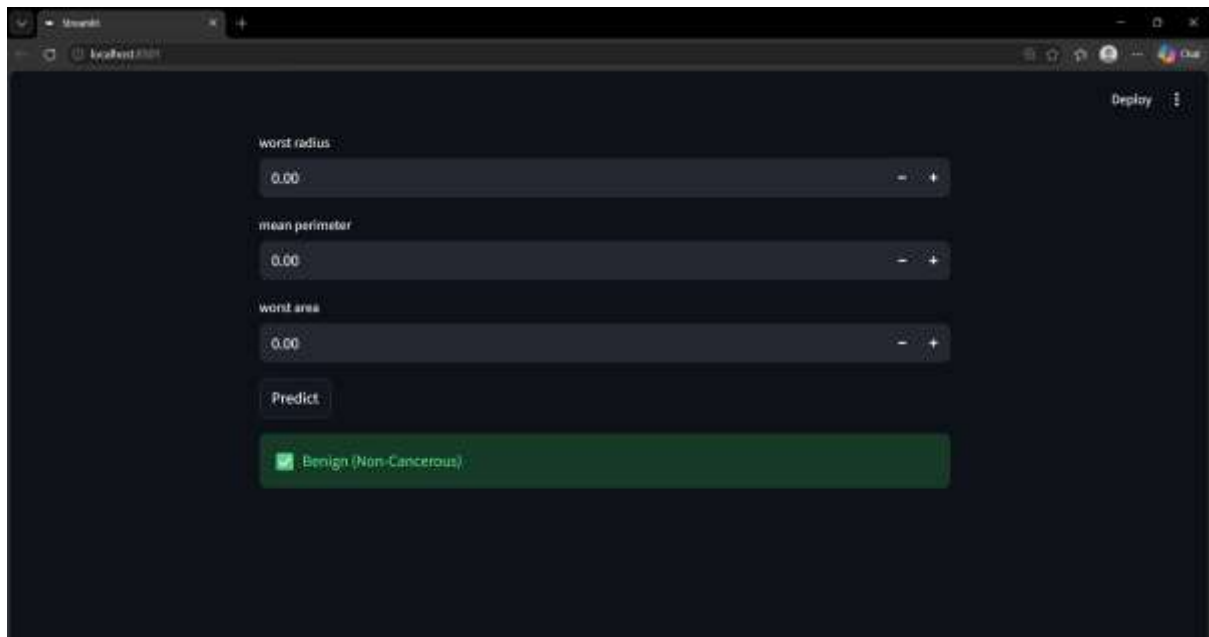
- Label: "worst concave points", Value: "7.99"
- Label: "worst perimeter", Value: "9.00"
- Label: "mean concave points", Value: "6.00"
- Label: "worst radius", Value: "5.00"
- Label: "mean perimeter", Value: "4.00"



A screenshot of the same web browser displaying the app after a prediction. The input fields now show the following values:

- Value: "9.00" (Label: "worst concave points")
- Label: "mean concave points", Value: "6.00"
- Label: "worst radius", Value: "5.00"
- Label: "mean perimeter", Value: "4.00"
- Label: "worst area", Value: "4.00"

Below the input fields is a "Predict" button. At the bottom, a red banner displays the prediction result: "⚠️ Malignant (Cancerous)".



Project 3:

Wine Classification

Problem Definition

Classifying wines based on their chemical properties is important in quality assessment and production analysis. However, manual classification is complex due to multiple influencing factors.

The problem is to develop a machine learning system that can accurately classify wines into different categories based on their chemical features.

Project Description

This project implements a supervised machine learning classification system that predicts the class of wine using its chemical attributes.

The workflow includes:

1. Training multiple supervised learning models
2. Comparing their performance
3. Selecting the best-performing model
4. Deploying the final model using a Streamlit application

After evaluation, Logistic Regression achieved the highest accuracy and was selected as the final model.

Feature scaling is applied to ensure better performance and consistency.

Tools & Libraries Used

Programming Language:

- Python

Libraries:

- Streamlit → Web app interface
- NumPy → Numerical computations
- Scikit-learn → Model training, scaling, evaluation
- Pickle → Saving and loading model, scaler, and features

Code Implementation:

```
import streamlit as st
```

```
import numpy as np
```

```
import pickle
```

```
model = pickle.load(open("wine_model.pkl", "rb"))
```

```
scaler = pickle.load(open("wine_scaler.pkl", "rb"))
```

```
features = pickle.load(open("wine_features.pkl", "rb"))
```

```
st.title("🍷 Wine Classification App")
```

```
input_data = []
```

```
for feature in features:
```

```
    value = st.number_input(feature, value=0.0)
```

```
    input_data.append(value)
```

```
input_data = np.array(input_data).reshape(1, -1)
```

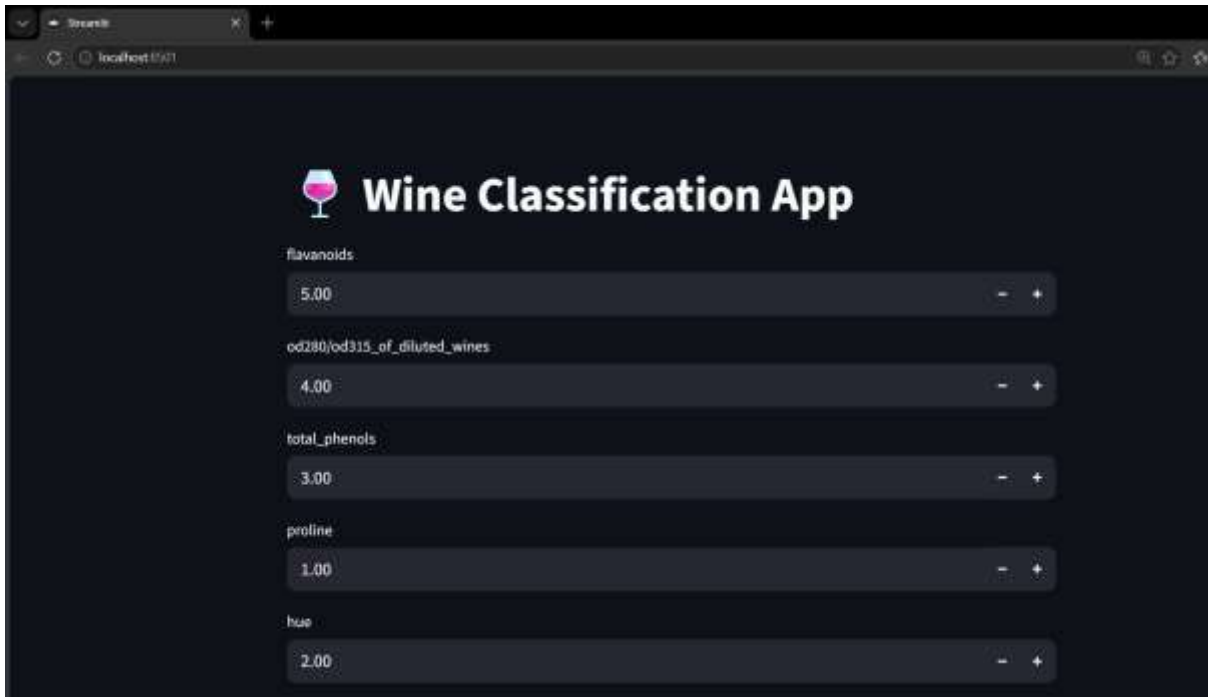
```
input_data = scaler.transform(input_data)
```

```
if st.button("Predict"):
```

```
    prediction = model.predict(input_data)
```

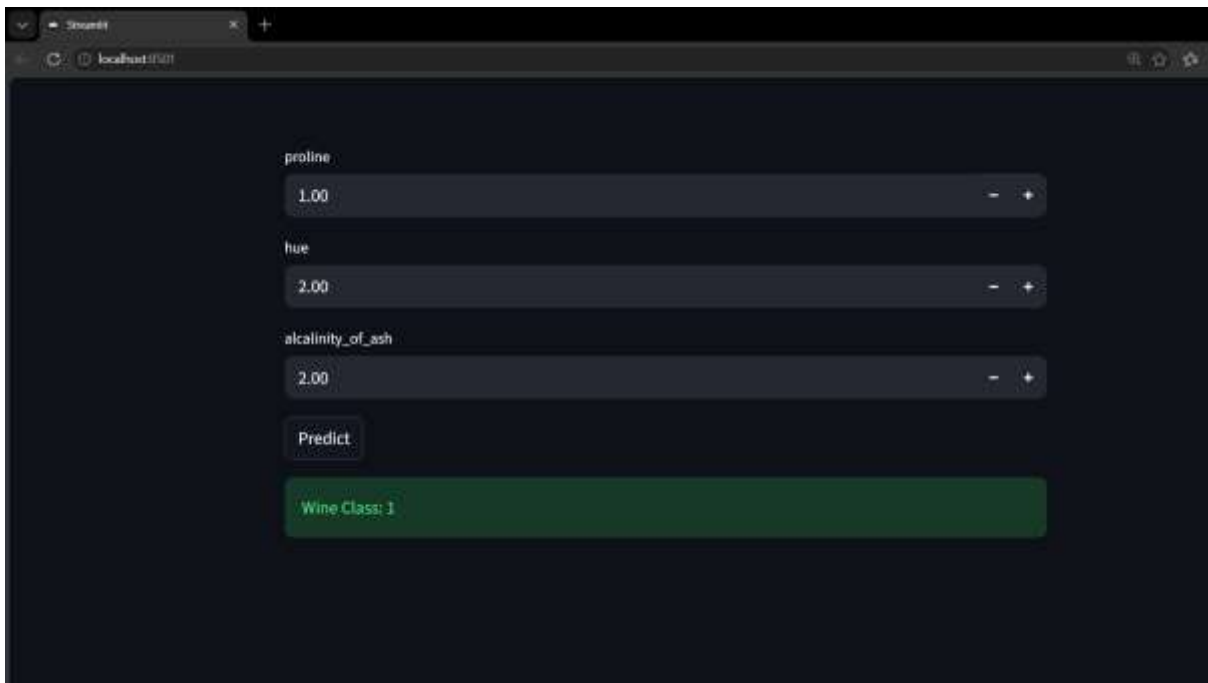
```
st.success(f"Wine Class: {prediction[0]}")
```

Output:



The screenshot shows a web browser window with the URL `localhost:5171`. The page has a dark theme and is titled "Wine Classification App" with a wine glass icon. It contains five input fields, each with a label, a value, and minus/plus buttons:

- flavanols: 5.00
- od280/od315_of_diluted_wines: 4.00
- total_phenols: 3.00
- proline: 1.00
- hue: 2.00



The screenshot shows the same web browser window after a prediction. The input fields for "proline" and "hue" are visible with values 1.00 and 2.00 respectively. Below them is an "alcalinity_of_ash" field with a value of 2.00. A "Predict" button is present, and below it, a green box displays the result: "Wine Class: 1".

Project 4:

Titanic Survival Prediction

Problem Definition

Predicting whether a passenger survived the Titanic disaster is a classic classification problem. Survival depends on multiple factors such as age, gender, ticket class, and fare.

The challenge is to develop a machine learning model that can accurately predict survival outcomes based on passenger data.

Project Description

This project implements a supervised machine learning classification system to predict whether a passenger survived or not.

The workflow includes:

1. Training multiple supervised learning models
2. Comparing their performance
3. Selecting the best-performing model
4. Deploying the final model using a Streamlit web application

After evaluation, Logistic Regression achieved the highest accuracy and was selected as the final model.

The system also includes categorical encoding and feature scaling to improve prediction performance.

Tools & Libraries Used

Programming Language:

- Python

Libraries:

- Streamlit → Web application interface
- NumPy → Numerical computations
- Scikit-learn → Model training, preprocessing, evaluation
- Pickle → Model and scaler storage

Code Implementation:

```
import streamlit as st

import numpy as np

import pickle

model = pickle.load(open("titanic_model.pkl", "rb"))
scaler = pickle.load(open("titanic_scaler.pkl", "rb"))

st.title("🚢 Titanic Survival Prediction")

pclass = st.selectbox("Passenger Class", [1,2,3])
sex = st.selectbox("Sex", ["Male", "Female"])
age = st.number_input("Age", value=25)
sibsp = st.number_input("Siblings/Spouses", value=0)
parch = st.number_input("Parents/Children", value=0)
fare = st.number_input("Fare", value=50.0)
embarked = st.selectbox("Embarked", ["C", "Q", "S"])

# Encoding
sex = 1 if sex == "Male" else 0
embarked_map = {"C":0, "Q":1, "S":2}
embarked = embarked_map[embarked]

input_data = np.array([[pclass, sex, age, sibsp, parch, fare, embarked]])
input_data = scaler.transform(input_data)

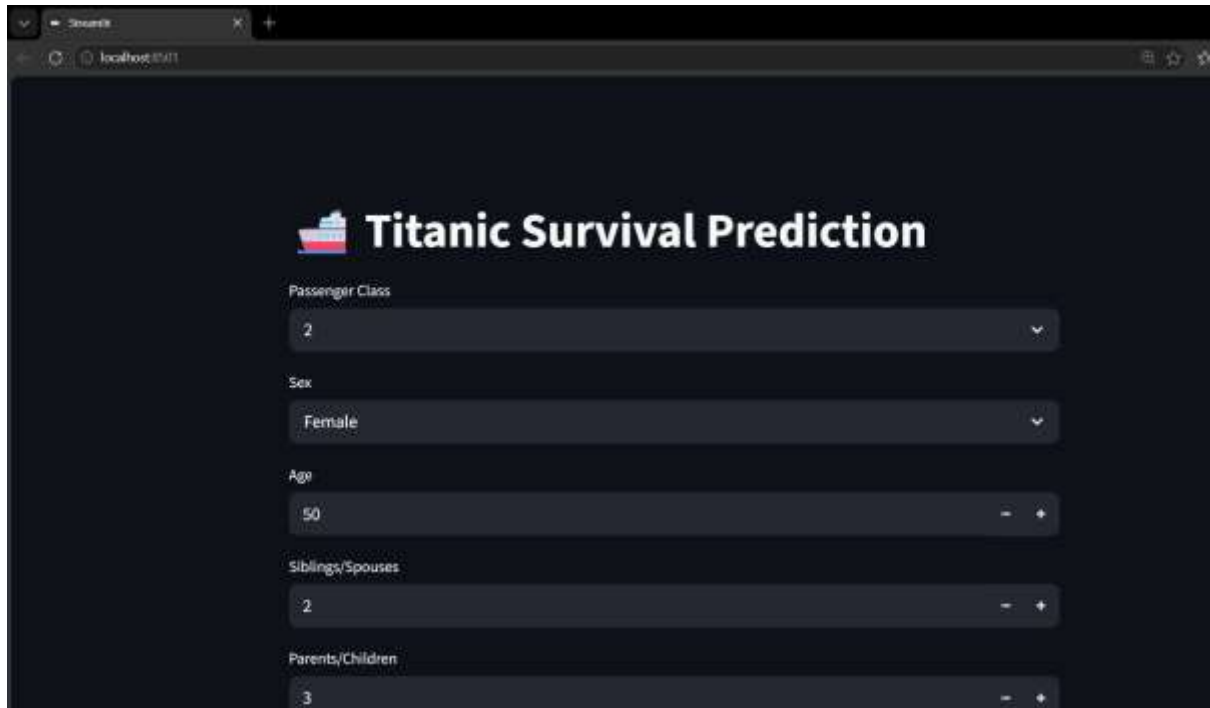
if st.button("Predict"):
    prediction = model.predict(input_data)

    if prediction[0] == 1:
        st.success("✅ Survived")
```

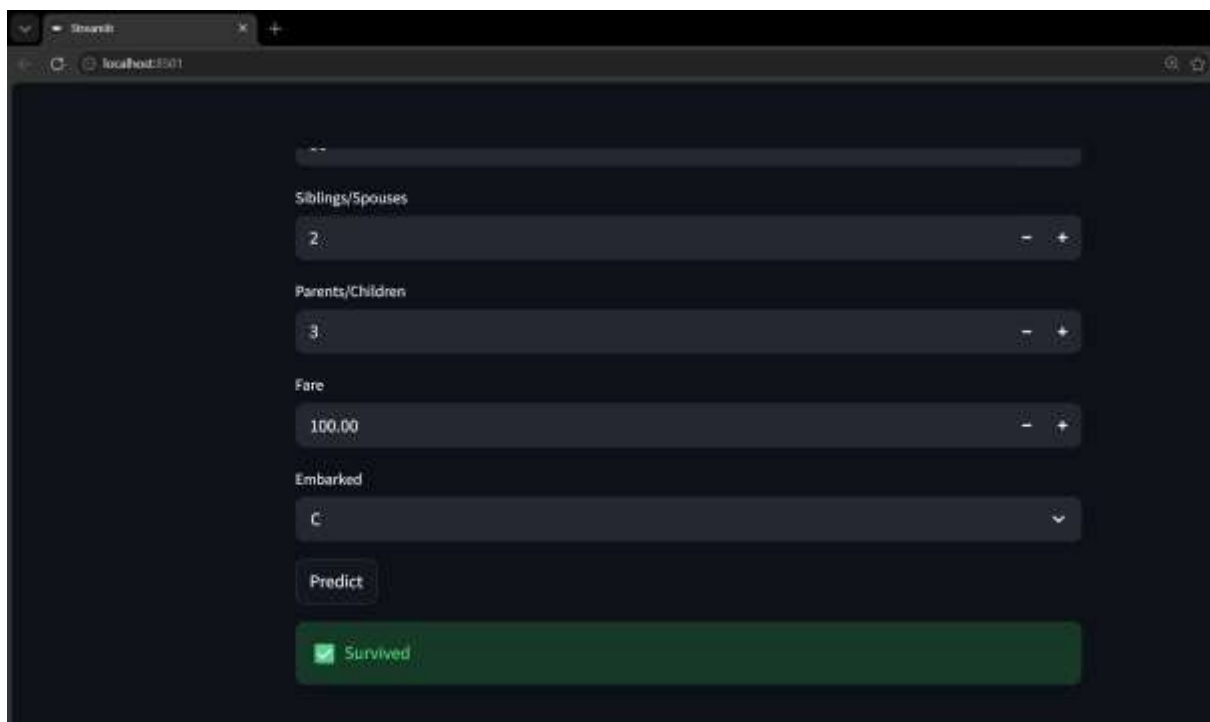
else:

```
st.error("❌ Not Survived")
```

Output:



The screenshot shows a web browser window with the URL `localhost:5001`. The page has a dark theme and is titled "Titanic Survival Prediction" with a ship icon. It contains several input fields: "Passenger Class" (dropdown menu with value 2), "Sex" (dropdown menu with value Female), "Age" (numeric input with value 50 and +/- buttons), "Siblings/Spouses" (numeric input with value 2 and +/- buttons), and "Parents/Children" (numeric input with value 3 and +/- buttons).



This screenshot shows the same web application after a prediction. The "Siblings/Spouses" field now has a value of 2, and "Parents/Children" has a value of 3. Below these fields, there is a "Fare" input field with the value 100.00 and an "Embarked" dropdown menu with the value C. A "Predict" button is visible. At the bottom, a green bar displays the result: "Survived" with a green checkmark icon.

Streamlit

localhost:8501

Titanic Survival Prediction

Passenger Class

3

Sex

Female

Age

50

Siblings/Spouses

2

Parents/Children

3

Streamlit

localhost:8501

50

Siblings/Spouses

2

Parents/Children

3

Fare

100.00

Embarked

C

Predict

✖ Not Survived